

Genetic Algorithm for the Traveling Student

Software Design Document

Charles Walker

December 11, 2023

CS 225, Fall 2023

Embry-Riddle Aeronautical University

Daytona Beach campus

1 Aerospace Boulevard

Daytona Beach, FL 32114

INTRODUCTION:

This Software Design Document (SDD) describes the Genetic Algorithm for the Traveling Student (GATS) program. GATS is designed to produce solutions to the combinatorial optimization problem, the traveling salesman problem (TSP), using a genetic algorithm (GA) designed to minimize the computations needed to produce a solution. GATS allows the user to alter variables in the GA, including but not limited to the size of the population produced, desired fitness level, elitism percentage, and mutation chance. To allow the user to test various ways to produce a solution, GATS saves results from previous computations. Thus, allowing the user to experiment and find faster ways to reach a solution while learning more about heuristic search algorithms.

GATS was created for a CS225 project but intended to be modified and updated later to learn more programming methods and ways to modify user interfaces.

GLOSSARY:

1. Chromosome: A biological analogy that represents the traits that make the individual [1].
2. Crossover rate: A biological analogy representing the percentage of individuals in a population that will produce offspring to form the next generation [1].
3. Elitism: A biological analogy, this is a process where individuals who have the highest fitness scores are selected for mating with the exception that they survive unaltered into the next generation [1].
4. Fitness rate: A biological analogy representing how adapted an organism is to its environment [1].
5. Gene: A biological analogy representing the building blocks of the individual's chromosome [1].
6. Genetic Algorithms: A branch of evolutionary algorithms designed primarily for optimization problems that emulate the natural processes of biological evolution over multiple generations of a population [1].
7. Hop: Used to define a measured distance between two cities.
8. Mutation: A biological analogy, this is a process of randomly selecting offspring and one of its genes to change [1].
9. Population Size: This is the number of individuals in the GA's population in any one generation [1].

PROBLEM DESCRIPTION:

TSP: The TSP is a problem representing a traveling salesman who must determine the shortest route to visit multiple cities only once before returning home, which is also the starting point.

The TSP used in GATS starts at the Embry-Riddle Aeronautical University (ERAU) Daytona Beach campus (DBC) and attempts to determine the shortest route to reach every state capital within the contiguous United States, visiting every city only once before returning to the DBC. This early version of GATS does not include roads and uses a straight-line approach to each destination.

Given that there are 48 cities with the starting location and ending location being the DBC, there are 49 total trips used to create a single route. Each trip represents a gene, and 49 genes create the route that will act as the chromosome for each individual in the population. The combination of trips is given by the equation $\frac{49!}{2! \cdot 47!} = 1176$. All 1176 measured distances between each of the 49 locations are stored in an external file.

The total number of routes that can be made from these trips is quantified using the equation $(n-1)!$ where “n” is the number of cities and minus one is added to account for the starting city, which is also the return city. After each trip, or gene, is added to the chromosome, the next gene selected has $(n-1-j)!$ fewer options. With 49 genes needed to make a chromosome, this leaves $48!$ or 1.24139×10^{61} possible chromosome combinations.

A GA will typically follow the process shown in the figure below.

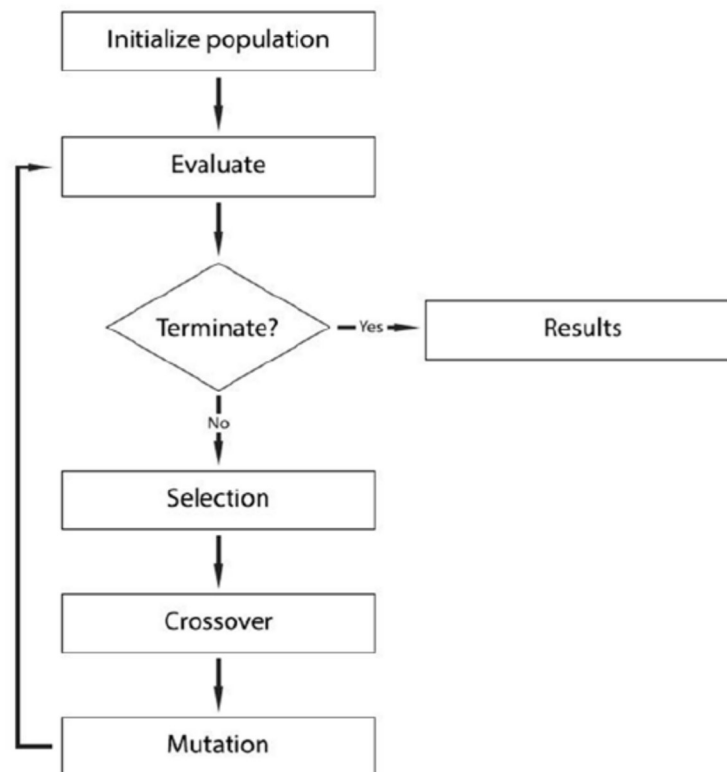


Fig. 1. Image of a typical genetic algorithm cycle. Adapted from [1].

The GATS follows the above diagram to search for better-optimized routes.

1. Initial Population: After the user has made the necessary choices, GATS randomly creates the initial population.
2. Evaluate: GATS evaluates a fitness score for each route in the population by adding the total distances of each trip in a chromosome. The shorter the total distance, the better the fitness score for that individual.
3. Terminate: If the user chooses to terminate, whether manually or by a predetermined method, GATS checks for that in this step. If GATS stops here, the results are recorded in an external file and printed on the screen.
4. Selection: GATS chooses what percentage of chromosomes carry on to the next generation and what percentage of chromosomes will be replaced based on the user's selected percentage of elites.
5. GATS applies the crossover method, or mating, to the chosen individuals. The best fitness scores will go through the elitism mating process, while the rest of the individuals who are selected for crossover will go through the normal process of mating and be replaced by their offspring.
6. GATS uses a user-chosen percentage to select offspring that will mutate a random gene and adjust the other genes to ensure every city is visited only once and the DBC remains the start and end point. GATS returns to step 2 and starts the process over again.

The following are termination methods the user can select to dictate when the algorithm stops before starting it:

1. Maximum number of generations.
2. A solution that meets the fitness threshold has been found.
3. Manual stoppage.

The following are variables in the algorithm that the user can manually adjust before running the program:

1. Population size.
2. Mutation chance.
3. Elite percentage (percentage of elites selected in each generation).
4. Use external file to initialize the population with. This allows the user to start from a saved stopping point.

PROBLEM SOLUTION:

Each chromosome of an individual in GATS is represented by a one-dimensional array containing 49 elements, one for each destination in a route. Two key points in GATS are how it interprets the chromosomes and how it manipulates the chromosomes by following two organizational rules.

1. GATS interprets the chromosome as an array with every element a number, 0 through 48, that creates one-half of a hop (or gene), with the entire chromosome creating 98 hops. With all 49 numbers representing the required destinations, each chromosome must maintain all 49 regardless of the order when they are manipulated. The first element read acts as the tail (starting destination of the hop), with its adjacent element acting as the head (ending destination in the hop). For example, element 2 would be a tail, and element 3 would be the head. Element 0 represents the DBC which will always be the first tail and last head in this process to simulate beginning and ending at the starting point. With element 0 always acting as the head of the last hop, it is the only exception where the head is not the adjacent element.
2. The second rule is that each number, 0 through 48, can only appear once in the chromosome. When new chromosomes are made through mating, the chromosome must be examined and adjusted as needed to ensure this rule is followed, or else a route may contain a destination that is visited twice while skipping one of the other required destinations creating an invalid solution and wasting computations.

The offspring creation and mutation process must follow the two previously mentioned rules. Each pair of parents creates offspring that are a mixture of each parent. From here on out in the document, when referring to the creation of new offspring and individuals, it means that new chromosome arrays are being created and replacing the old chromosome arrays that are held by individuals in the individual population array. With all chromosomes starting with the number 0 in the first element, each child is initially created with elements 1 through 24, a subset of one parent, and elements 25 through 48, a subset of a second parent. The two offspring-creation processes are crossover and elite mating, which will be discussed later.

After the children's chromosomes are initialized, they undergo an adjustment process that compares the first half of the first child's chromosomes to its second half to find repeating numbers. Once a repeating number is found, the second child's first half of chromosomes are compared to its second half until a repeating number is found in it. Once a repeating number is found in each child, they are swapped. By the end of the process, each child now contains all required numbers, a large unaltered subset of one parent, and partial subsets of the second

parent. This is possible due to each child containing one-half of each parent's chromosome, ensuring that each child's missing numbers are contained by the other child. The code example is given below.

```
public void fixOffspring(int[] kid1, int[] kid2) {
    int temp;
    boolean stop;
    for (int i = HGENES+1; i < TOTGENES; i++) {
        for (int j = 1; j < HGENES+1; j++) {
            if (kid1[i]==kid1[j]) {
                stop = false;
                for (int k = HGENES+1; k < TOTGENES && stop == false; k++) {
                    for (int l = 1; l < HGENES+1 && stop == false; l++) {
                        if (kid2[k] == kid2[l]) {
                            temp = kid2[k];
                            kid2[k] = kid1[i];
                            kid1[i]=temp;
                            stop = true;
                        }
                    }
                }
            }
        }
    }
}
```

In the code above, the method is given the arrays of the two new offspring as arguments. The variable "TOTGENES" represents the total number of elements in the chromosome array. The variable "HGENES" is given by the equation $(TOTGENES-1)/2$. This allows for the loops to exclude the first element and divide the chromosomes by half for comparison. The "temp" variable holds a temporary value to swap elements between the children's arrays. The inner two for-loops use the "stop" variable to end once a repeating number is found to reduce unneeded loops from occurring.

Below is an example of two arrays going in and coming out of the method.

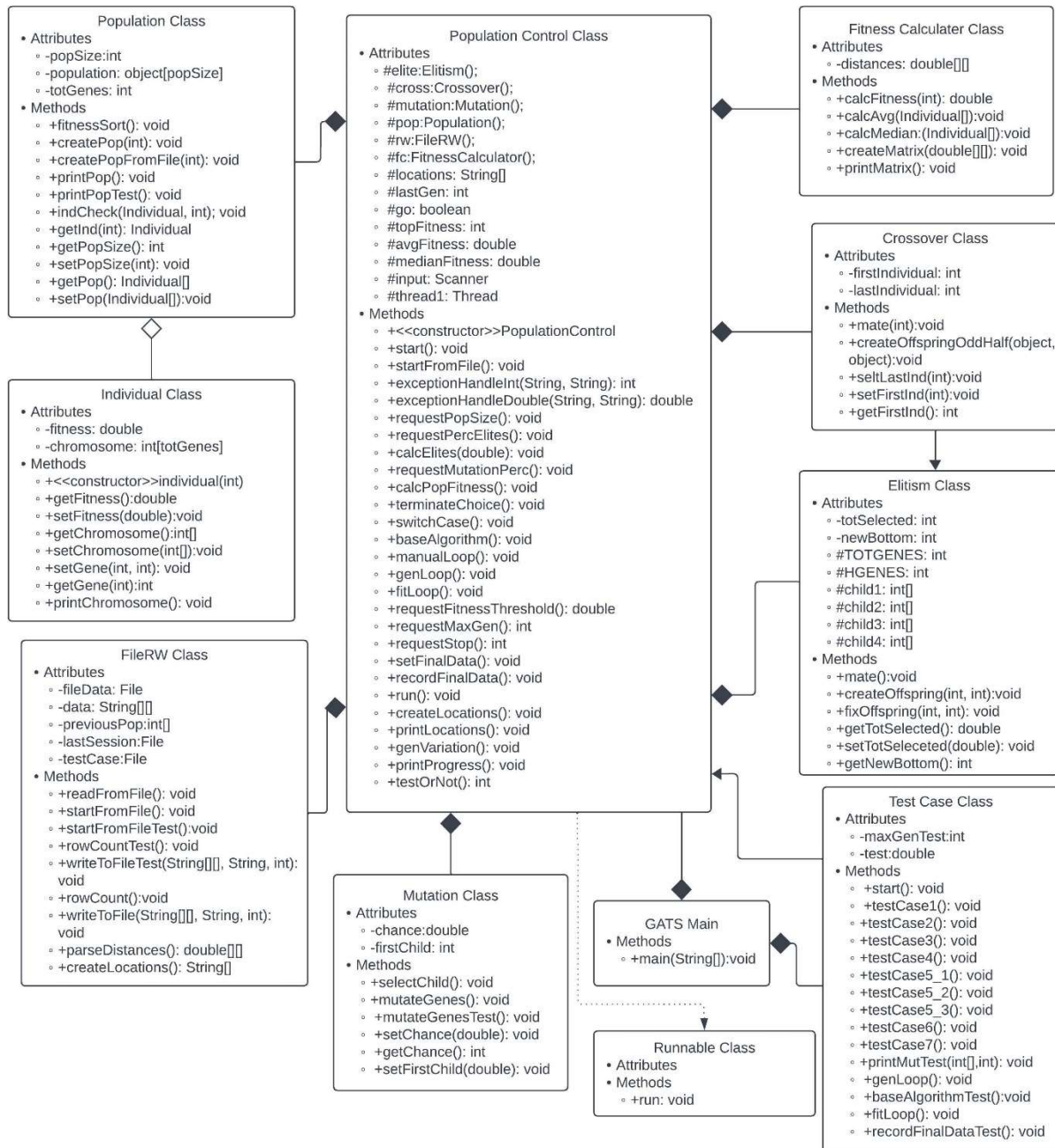
```
child1 old:
[0,28,17,35,7,10,45,23,34,1,48,20,12,13,2,3,43,47,26,44,11,8,36,29,14,35,11,4
8,44,38,36,31,2,15,1,10,21,5,12,7,23,3,25,43,32,26,34,9,14]
child2 old:
[0,8,22,41,13,37,17,30,20,42,39,18,46,16,6,27,4,45,19,47,40,29,28,33,24,46,27
,6,40,33,18,31,21,39,25,24,37,9,15,4,32,5,22,38,41,16,19,42,30]
child1 new:
[0,28,17,35,7,10,45,23,34,1,48,20,12,13,2,3,43,47,26,44,11,8,36,29,14,46,27,6
,40,38,33,31,18,15,39,24,21,5,37,4,22,41,25,16,32,19,42,9,30]
child2 new:
[0,8,22,41,13,37,17,30,20,42,39,18,46,16,6,27,4,45,19,47,40,29,28,33,24,35,11
,48,44,36,2,31,21,1,25,10,12,9,15,7,32,5,23,38,3,43,26,34,14]
```

The mutation process follows the same rules as the offspring creation process but only requires the random selection and swapping of two chromosomes.

Once the program starts, GATS creates a 49x49 two-dimensional array using a file containing the measurements between all destinations. This array is used to calculate the fitness scores of each individual without having to recalculate the distances each time a fitness score is determined. Each of the 98 hops in a chromosome, represented by the 49 genes, is also the 2-dimensional coordinates of that hop's distance location in the two-dimensional array.

GATS consists of eleven classes, including two extended classes. File operations are handled by the FileRW class, automated test cases are handled by the Test Case class, and multithreading is handled by the Runnable class. All three of which will not be covered in this document.

GATS is controlled by the main class, GATS Main. GATS Main creates an instance of the Population Control class and the Test Case class that extends the Population Control class. GATS Main calls the Population Control class's start() method to start the algorithm. The Population Control class creates six instances of six other classes: the Elitism class, the Crossover class, the Mutation class, the Fitness Calculator class, the Population class, and the File RW (Read Write) class. The Population class creates an array of instances of the Individual class. The Population Control class uses various methods to interact with the user and retrieve information needed to run the algorithm. Due to the complexity and reliance on other classes, the Population Control class will be described last. The ULM diagram is given in the figure below.



Fitness Calculator Class:

This class is used to calculate the fitness of each individual, the average fitness of the population, and the median fitness of the population. The `calcFitness()` method calculates the fitness of each individual and assigns it to a variable contained in that individual. The `calcAvg()` method calculates and returns the average fitness level, and the `calcMedian()` calculates and

returns the median fitness level. The median and average values are stored in variables held by the Population Control class. The createMatrix() method creates the distance matrix that holds the distances between each location. This array is used to calculate the fitness of each individual in the population, and those fitness scores are used to calculate the average and mean of the population.

Elitism Class:

The purpose of the Elitism class is to maintain the fittest of the population to prevent the highest fitness scores of the population from being lost in the mating process. The Elitism class also acts as the basis for the mating process for which the crossover class extends. The Elitism class uses the mate() method to call on all other methods within the class to perform all functions. This class tracks the total number of individuals to be selected for elitism using the attribute "totSelected" which holds a value the user chooses at the program's start.

The "totSelected" value is the number of elites to be used for the elite mating process. The totSelected value determines the number of loops the mate() method completes before stopping. The totSelected value is used in conjunction with the population array to determine who is chosen to survive to the next generation while also creating offspring. The population array remains organized from most to least fit by the Population Control class. This allows the mate() method to select each pair of parents quickly by starting with the fittest and descending down the list until it reaches the "totSelected" value. The last individual used in the population is recorded by the "count" variable that is initialized as one less of the population's size and decrements by four every iteration. After the loop is finished, the "count" value is assigned to the "newBottom" variable. This number tells the Crossover class where to start placing its children while the value of "totSelected" plus one is used as the starting point of the first parent for crossover. The algorithm for mating is given below.

```
public Individual[] mate(Individual[] population) {
    int count = population.length-1;
    for(int i = 0; i < totSelected; i+=2) {
        createOffspring(population[i].getChromosome(),
            population[i+1].getChromosome());
        population[count].setChromosome(child1);
        population[count-1].setChromosome(child2);
        population[count-2].setChromosome(child3);
        population[count-3].setChromosome(child4);
        count -=4;
        if(count == (population.length-(totSelected*2)-1)) {
            newBottom = count;
        }
    }
    return population;
}
```

The createOffspring() method will produce four new sets of chromosome arrays for the new offspring. Four offspring are created to increase the likelihood of passing on the more favorable subsets of genes that are responsible for the parents' higher fitness ratings. In the example code below, p1 and p2 are the parents. "TOTGENES" and "HGENES" are the same as the previous code example.

```
public void createOffspring(int[] p1, int[] p2) {
    child1 = new int[TOTGENES];
    child2 = new int[TOTGENES];
    child3 = new int[TOTGENES];
    child4 = new int[TOTGENES];
    child1[0] = 0;
    child2[0] = 0;
    child3[0] = 0;
    child4[0] = 0;
    for (int i = 1; i < HGENES+1; i++) {
        child1[i] = p1[i];
        child2[HGENES+i] = p1[HGENES+i];
        child1[HGENES+i] = p2[HGENES+i];
        child2[i] = p2[i];
        child3[i] = p1[HGENES+i];
        child4[HGENES+i] = p1[i];
        child3[HGENES+i] = p2[i];
        child4[i] = p2[HGENES+i];
    }
    fixOffspring(child1, child2);
    fixOffspring(child3, child4);
}
```

The offspring of elites replace the lowest-scoring individuals' chromosome arrays in the population rather than replacing the parent's chromosome arrays. The program is designed to have at least two elites in each generation and always an even amount. If the percentage of requested elites chosen produces a fractional number due to the size of the requested population, the number will be truncated to make an integer. If that integer is odd, one will be added to the number to make it an even number of elites. This process is performed by the Population Control class prior to the "totSelected" value being set.

Crossover Class:

This Crossover class is an extension of the Elitism class. The attributes "firstIndividual" and "lastIndividual" are added to track where the overridden mate() method starts pulling from the population. The mate() method acts similarly to the Elitism class's mate() method in that it calls

on all of the other methods, but it is modified so that two of the four offspring replace the parents while the other two offspring replace two of the lower scoring individuals at the end of the array. The variable “count” is set to a value one less than the “lastIndividual” to show the last element from the bottom of the population array that was replaced. As the Crossover class replaces four individuals at a time, it updates the “count” and “i” variables in the for-loop to be two less and more, respectively. This process prevents the replacement of the wrong indexes in the population. The `mate()` method selects the top half of the remaining population that wasn’t selected for elitism. This subset of individuals will replace themselves and those not selected for crossover with their offspring.

There is a chance, depending on the size of the population and number of elites selected, that the last iteration of the `mate()` method’s for-loop will leave two individuals untouched. When this happens, the difference between “count” and “i” will be one. An if statement is placed after the `mate()` method’s for-loop with the condition of “count” minus “i” equals one. Inside the if statement, the method `createOffspringOddHalf()` is called. This method is similar to `createOffspring()` except that it only creates two offspring that replace the two given parents.

Mutation Class:

This class is responsible for delaying the convergence of solutions over multiple generations by mimicking the natural evolutionary process of adding genetic diversity to a population. Otherwise, the end solution would only consist of the original genes of the first population, and the solutions would plateau quickly. While the offspring creation method used by GATS does allow for variations from the parents’ original genes, this allows for more varied solutions.

The “chance” variable is a user-given value used to determine each offspring’s chance at mutation upon creation. The `selectChild()` method rolls a chance, based on the percentage given, for each new offspring in the population. If selected, the `mutateChild()` method selects two random elements in the offspring’s chromosome array to swap. Following the original rules stated prior, element index 0 is left unaltered. The first child in the population is set to the same index number used by the Crossover class to determine its first parent. This way the Mutation Class knows which individuals in the population to role for a chance of mutation.

Individual Class:

This class acts as the blueprint for all individuals in the population. It contains two attributes and a constructor. The “fitness” variable holds the individual’s fitness value assigned by the Fitness Calculator class. The `chromosome[]` array is an array of integers that holds the individual’s destination order that creates its chromosome. The constructor method for this class takes an integer for an argument that it uses to create its chromosome array.

Population Class:

This class is responsible for initializing and creating the population of individuals. Its “popSize” variable is a value given by the user to determine the size of the population. Every Individual is initialized with a chromosome array containing all 48, numbers in order, by the createPop() method and is then mixed by selecting two genes and swapping them 48 times to maximize genetic variation. The indCheck() method is used in conjunction with the printPpo() method for testing purposes to ensure that every chromosome contains all numbers. The fitnessSort() method arranges the population by fitness rating, from smallest to largest.

Population Control Class:

When the GATS Main class calls on the Population Control class’s start() method, the program begins. After calling the start() method, the user is asked whether or not to start from a file. If the user chooses yes, the file is read to create the population, where the data is put into an array and passed to the Population class to be converted into the population. If the user chooses no, the user is prompted to give a population size by calling on the requestPopSize() method. The size is passed to the Population class to create the initial population. The user is then asked to provide a percentage for desired elites and mutation chance through the requestPercElites() and requestMutationPerc() methods. The calcPopFitness() method is called to assign fitness levels to the population and the population class’s fitnessSort() method is called to sort the population to prepare the GA process. Once sorted, the top-scoring fitness is assigned to the “topFitness” attribute.

The final step, before the GA begins its loop, is deciding the termination method and requesting parameters from the user depending on the method chosen. The terminateChoice() method prompts a menu and requests the choice. The switchCase() method is then called to start the process based on the user’s decision. The following list contains the steps for each method.

Manual Stoppage:

If the user selects option one, no further input is requested. The manualLoop() method is called and runs until the user presses enter. While running, the screen is constantly refreshed to display the current generation of the population, the best solution found, and a message telling the user to press enter at any time to stop. If the user presses enter, the requestStop() method is called to verify if the user wants to stop. If not, the algorithm continues to run. If yes, the algorithm stops, saves the final data to “Last Session.csv” and prompts the user to press enter once more to end the program.

Maximum Number of Generations:

If the user selects option two, the `genLoop()` method is called. The user is prompted to pick a number greater than one for the maximum number of generations. Once the user enters a number, the algorithm runs in a loop, adding one to the “lastGen” attribute every iteration until the given stopping point matches the “lastGen” attribute or the user presses enter to manually stop it. The user will still have the option to continue again until the maximum generation is reached. Once the algorithm stops, the population data is saved to “Last Session.csv” and the user is prompted to press enter to end the program.

Fitness Level Threshold:

If the user selects option three, the `fitLoop()` method is called, and the user is prompted to provide a threshold between zero and eighty thousand. After receiving the threshold, the program runs until either the user presses enter, or a solution is found that is less than or equals the threshold. Once a solution that meets the threshold is found, the program saves the data and prompts the user to press enter to end the program.

All termination methods utilize the `baseAlgorithm()` method that follows the steps in Figure 1 above, with each loop method acting as the decision factor. The `baseAlgorithm()` method is given below.

```
public void baseAlgorithm() {
    pop.setPop(elite.mate(pop.getPop()));
    cross.setFirstInd((int) (elite.getTotSelected()+1));
    cross.setLastInd((pop.getPop().length-
((int)elite.getTotSelected()*2)-1));
    pop.setPop(cross.mate(pop.getPop()));
    mutation.setFirstChild(cross.getFirstInd());
    calcPopFitness();
    pop.fitnessSort();
    topFitness = pop.getInd(0).getFitness();
    lastGen++;
    genVariation();
    printProgress();
}
```

REFERENCES:

- [1] L. Jacobson and B. Kanber, *Genetic Algorithms in Java Basics*. Berkeley, CA: Apress, 2015.